

UNIVERZITA PARDUBICE

FAKULTA ELEKTROTECHNIKY A
INFORMATIKY

BAKALÁŘSKÁ PRÁCE

2025

Egor Razboinikov

Univerzita Pardubice
Fakulta elektrotechniky a informatiky

Digitalizace evidence pracovní doby
Bakalářská práce

2025

Egor Razboinikov

Univerzita Pardubice
Fakulta elektrotechniky a informatiky
Akademický rok: 2023/2024

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

(projektu, uměleckého díla, uměleckého výkonu)

Jméno a příjmení:	Egor Razboinikov
Osobní číslo:	I21219
Studijní program:	B0688A140009 Informační technologie
Téma práce:	Digitalizace evidence pracovní doby
Zadávací katedra:	Katedra informačních technologií

Zásady pro vypracování

Dokument evidence pracovní doby vyučujícího vyžaduje provázanost rozvrhu s úvazkem vyučujícího při respektování času přestávek. Cílem bakalářské práce je navrhnout, vytvořit, odzkoušet a předat funkční multiplatformní aplikaci jejíž vstupy jsou: rozvrh vyučujícího, čas na oběd, plán dovolených, státní svátky, školní volna a další. Požadovaným výstupem je PDF dokument (tabulka) se všemi náležitostmi potřebnými pro odevzdání. Grafické rozhraní poskytne pohodlné zadávání, umožní dodatečné ruční modifikace a zobrazování náhledu před vygenerováním PDF dokumentu.

Rozsah pracovní zprávy: **min. 30 stran**
Rozsah grafických prací:
Forma zpracování bakalářské práce: **tištěná**

Seznam doporučené literatury:

PŠENČÍKOVÁ, J. Algoritmizace. Kralice na Hané, Computer Media, 2009.
MAREŠ, Martin a VALLA, Tomáš. Průvodce labyrintem algoritmů 2. vydání. Praha: CZ.NIC, 2022. ISBN 978-80-88168-63-8
PECINOVSKÝ, Rudolf. OOP: naučte se myslet a programovat objektivě. Brno: Computer Press, 2010. ISBN 978-80-251-2126-9.

Vedoucí bakalářské práce: **Ing. Radek Matoušek**
Katedra informačních technologií

Datum zadání bakalářské práce: **15. prosince 2024**
Termín odevzdání bakalářské práce: **16. května 2025**

prof. Ing. Petr Doležel, Ph.D. v.r.
děkan

L.S.

Ing. Jan Panuš, Ph.D. v.r.
vedoucí katedry

Prohlášení autora

Prohlašuji:

Práci s názvem Digitalizace evidence pracovní doby jsem vypracoval samostatně. Veškeré literární prameny a informace, které jsem v práci využil, jsou uvedeny v seznamu použité literatury.

Byl jsem seznámen s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon), ve znění pozdějších předpisů, zejména se skutečností, že Univerzita Pardubice má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je Univerzita Pardubice oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladů, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

Beru na vědomí, že v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů, a směrnicí Univerzity Pardubice č. 7/2019 Pravidla pro odevzdávání, zveřejňování a formální úpravu závěrečných prací, ve znění pozdějších dodatků, bude práce zveřejněna prostřednictvím Digitální knihovny Univerzity Pardubice.

V Pardubicích dne 03. 05. 2025

Egor Razboinikov v.r.

PODĚKOVÁNÍ

Tady chtěl bych poděkovat svému vedoucímu Ing. Radek Matoušek za hezké vedení bakalářské práce a rychle odpovědi. Také rád bych poděkoval svým rodičům a kamarádům za jejich podporu po celou dobu studia.

ANOTACE

Cílem této bakalářské práce je návrh, implementace a ověření funkčnosti multiplatformní aplikace pro digitalizaci evidence pracovní doby vyučujících. Aplikace umožňuje automatizované zpracování dat, jako jsou rozvrh hodin vyučujícího, stanovené přestávky na oběd, plánované dovolené, státní svátky a školní volna. Výsledným výstupem je detailní PDF dokument, který splňuje formální požadavky fakulty a lze jej snadno upravovat prostřednictvím intuitivního grafického rozhraní. V práci je popsána použitá metodika, výběr vhodných technologií a shrnuty výsledky praktického ověření aplikace.

KLÍČOVÁ SLOVA

C#, .NET, AVALONIA, MVVM, .AXAML, desktopová aplikace, rozvrh hodin, evidence pracovní doby, generování PDF, automatizace.

TITLE

Digitalization of Working Time Records

ANNOTATION

The aim of this bachelor thesis is to design, implement, and verify a functional cross-platform application for digitizing teachers' working time records. The application automates data processing, including teachers' timetables, designated lunch breaks, vacation schedules, public holidays, and school breaks. The output is a detailed PDF document that meets the faculty's formal submission requirements and can be easily modified via an intuitive graphical interface. The thesis describes the applied methodology, technology selection, and summarizes results of the practical evaluation of the application.

KEYWORDS

C#, .NET, WPF, MVVM, AVALONIA, desktop application, timetable, working time tracking, PDF generation, automation.

OBSAH

SEZNAM ILUSTRACÍ A TABULEK.....	10
SEZNAM ZKRATEK A ZNAČEK	11
TERMINOLOGIE	12
ÚVOD.....	13
1 TEORETICKÁ ČÁST	14
1.1 Cíle aplikace	14
1.2 Funkcionalita aplikace	14
1.2.1 Import rozvrhu ze STAG	14
1.2.2 Globální nastavení pracovních parametrů	14
1.2.3 Grafické rozhraní pro úpravu denních údajů	14
1.2.4 Automatické značení svátků a volných dnů	14
1.2.5 Generování PDF dokumentu	15
1.3 Programovací jazyky	15
1.3.1 Jazyk C#.....	15
1.3.2 Jazyk SQL.....	15
1.3.4 XAML (.axaml) v Avalonia UI	16
1.4 Databázové technologie	16
1.4.1 SQLite	16
1.5 Vývojové prostředí	17
1.5.1 Microsoft Visual Studio.....	17
1.5.2 SQLiteStudio	17
1.6 Frameworky	17
1.6.1 Avalonia UI.....	17
1.7 Architektura desktopové aplikace.....	19
1.7.1 Architektura MVVM	20
1.8 Knihovny	20
1.8.1 Knihovna Avalonia	21
1.8.2 Knihovna Avalonia.Desktop & Avalonia.Diagnostics	21
1.8.3 Knihovna Avalonia.ReactiveUI.....	21
1.8.4 Knihovna Avalonia.Themes.Fluent & Avalonia.Fonts.Inter	21
1.8.5 Knihovna Avalonia.Xaml.Behaviors & Avalonia.Xaml.Interactivity.....	21
1.8.6 Knihovna ReactiveUI	21
1.8.7 Knihovna Entity Framework Core & SQLite	21
1.8.8 Knihovna Nager.Date	22

1.8.9 Knihovna QuestPDF	22
1.8.10 Knihovna Ghostscript.NET	22
1.8.11 Knihovna MessageBox.Avalonia	22
2 PRAKTICKÁ ČÁST	22
2.1 Souborová struktura aplikace.....	22
2.1.1 Složka Behaviors	24
2.1.2 Složka Controls.....	24
2.1.3 Složka Converters	24
2.1.4 Složka Data	25
2.1.5 Složka Helpers	26
2.1.6 Složka Messages	27
2.1.7 Složka Models.....	27
2.1.8 Složka Models.....	27
2.1.9 Složka ViewModels	29
2.1.10 Složka Views	29
2.1.11 Složka Kořenové soubory projektu.....	29
2.2 Struktura databáze.....	30
2.2.1 Tabulka Events	30
2.2.2 Tabulka GlobalSettings.....	31
2.2.2 Tabulka UserSettings	31
2.3 Uživatelské rozhraní aplikace	31
2.3.1 Hlavní stránka aplikace.....	32
2.3.2 Dialogové okno pro vytvoření události	32
2.3.3 Vizuální prezentace pro den	33
2.3.3 Vizuální prezentace pro týden	34
2.3.3 Vizuální prezentace pro měsíc	34
2.3.3 Globální nastavení	35
2.3.3 Ukázka PDF	35
ZÁVĚR	36
POUŽITÁ LITERATURA	38
SEZNAM PŘÍLOH.....	38

SEZNAM ILUSTRACÍ A TABULEK

Obrázek 1: Architektura aplikace v .NET	18
Obrázek 2: Architektura MVVM	20
Obrázek 3: Souborová struktura aplikace	23
Obrázek 4: Ukázka kódu třídy AppDbContext.....	26
Obrázek 5: Hlavní stránka aplikace	32
Obrázek 6: Dialogové okno pro vytvoření	33
Obrázek 7: Vizuální prezentace pro den.....	33
Obrázek 8: Vizuální prezentace pro týden.....	34

Obrázek 9: Vizuální prezentace pro měsíc	34
Obrázek 10: Globální nastavení.....	35
Obrázek 11: Ukázka PDF	36

SEZNAM ZKRATEK A ZNAČEK

CSV – Comma-Separated Values

DB – Databáze

DI – Dependency Injection

EPD – Evidence Pracovní Doby

EF Core – Entity Framework Core

FK – Foreign Key

GUI – Graphical User Interface

UI – User Interface

JIT – Just-In-Time

MVVM – Model-View-ViewModel

ORM – Object-Relational Mapping

PDF – Portable Document Format

UI – User Interface

CLR – Common Language Runtime

XAML – Extensible Application Markup Language

RI – Run-Time Information

RPC – Remote Procedure Call

CI/CD – Continuous Integration / Continuous Deployment

TERMINOLOGIE

Avalonia UI:

Multiplatformní framework pro vývoj uživatelského rozhraní aplikací, který umožňuje tvorbu vizuálně konzistentních aplikací běžících na různých operačních systémech, jako jsou Windows, macOS a Linux, za pomoci XAML a programovacího jazyka C#.

Framework:

Sada nástrojů, knihoven a standardizovaných postupů, která usnadňuje proces vývoje softwaru poskytováním struktury a základních funkcionalit, umožňujících rychlou implementaci úloh bez nutnosti opakovaného vytváření stejného kódu.

Relační databáze:

Typ databáze ukládající informace v tabulkách s jasně definovanými vztahy (např. vazby jedna k jedné, jedna k mnoha), umožňující efektivní ukládání, vyhledávání a správu dat.

Programovací paradigma (MVVM):

Model-View-ViewModel představuje návrhový vzor programování, který jasně odděluje uživatelské rozhraní (View), datovou logiku (Model) a komunikační logiku (ViewModel), což vede k lepší udržovatelnosti a škálovatelnosti aplikace.

Uživatelské rozhraní (UI):

Vizuální a interaktivní část aplikace, navržená pomocí frameworku Avalonia UI, která umožňuje uživatelům jednoduše komunikovat s programem prostřednictvím grafických prvků, jako jsou tlačítka, formuláře, menu a další ovládací prvky.

Integrované vývojové prostředí (IDE):

Softwarový nástroj kombinující editor kódu, kompilátor, debugger a nástroje pro správu verzí, používaný při vývoji aplikací.

Systémy pro správu verzí (VCS):

Nástroje umožňující sledování změn kódu, uchování historie verzí, spolupráci mezi vývojáři a obnovu předchozích verzí kódu v případě potřeby.

ÚVOD

V každodenní praxi vysokoškolských administrativních oddělení se jednak bezchybné zaznamenávání odpracovaných hodin vyučujících setkává s vysokými nároky na přesnost a soulad s legislativou, a zároveň naráží na limity manuálních procesů. Excelovské šablony, do kterých zaměstnanci přepisují výstupy z IS STAG a úvazkové plány, se postupně plní desítkami sloupců a řádků, kde jediná chyba v časovém zápisu dokáže vyvolat řetězec oprav a nápravných kroků. Časové prodlevy při hledání a opravě chyb, kumulace nevyřízených zápisů a nemožnost okamžité aktualizace zpráv o fondu pracovní doby přispívají k rostoucí administrativní zátěži a ztěžují plánování výukových kapacit.

Pro odstranění těchto úzkých míst si bakalářská práce klade za cíl navrhnout, implementovat a ověřit multiplatformní desktopovou aplikaci „Digitalizace evidence pracovní doby“, která převádí vstupní data — rozvrh vyučujícího exportovaný ze STAG v CSV formátu. Pomocí frameworku Avalonia UI a architektury MVVM zajišťuje aplikace konzistentní uživatelské rozhraní na Windows i Linuxu, umožňuje automatické označení státních svátků a školních prázdnin, přidává funkce pro manuální úpravy časů a v reálném čase vykresluje náhled výsledné tabulky. Konečným produktem je validní PDF dokument, který přesně odpovídá měsíčním a týdenním fondům pracovní doby a upozorňuje na případná překročení nebo kolize.

V teoretické části práce je nejprve vymezen právní a organizační rámec vedení evidence pracovní doby na vysokých školách, včetně relevantních směrnic a zákonných ustanovení. Následuje popis principů digitalizace a srovnání nejpoužívanějších přístupů k vývoji desktopových aplikací, s důrazem na multiplatformní framework Avalonia UI a návrhový vzor Model-View-ViewModel. Dále se práce věnuje technologickým komponentám pro zpracování CSV souborů, ukládání dat v SQLite a metodám generování PDF dokumentů. Závěrečná část teorie rozebírá návrhové vzory a detailní algoritmus pro automatické rozložení pracovního fondu podle stanovených pravidel.

V praktické části je podrobně popsána struktura řešení — datový model, modul importu CSV, klíčové komponenty uživatelského rozhraní a generátor PDF. Součástí je také popis procesu testování na reálných datech vyučujících, měření výkonu a návrh možných rozšíření, například o jednotnou autentizaci (SSO), auditní logy či export do dalších formátů. Tím vzniká ucelené řešení, které významně snižuje administrativní náročnost a zvyšuje přesnost evidence pracovní doby na akademických pracovištích.

1 TEORETICKÁ ČÁST

Očividně je uvázaná dostatečně pevně, protože balónky skutečně neucházejí. To ale není nic zvláštního. Překvapit by však mohl fakt, že nikdo, snad krom toho, kdo balónky k obloze vypustil, netuší, jakou má ona stuha barvu. Je totiž tak lesklá, že za světla se v ní odráží nebe a za tmy zase není vidět vůbec. Když svítí slunce tak silně jako nyní, tak se stuha třpytí jako kapka rosy a jen málokdo vydrží dívat se na ni přímo déle než pár chvil. Jak vlastně vypadají ony balónky? Ptají se často lidé. Inu, jak by vypadaly – jako běžné pout'ové balónky střední velikosti, tak akorát nafouknuté. Červený se vedle modrého a zeleného zdá trochu menší, ale to je nejspíš jen optický klam, a i kdyby byl skutečně o něco málo menší, tak vážně jen o trošičku. Vítr skoro nefouká, a tak by se na první pohled mohlo zdát, že se balónky snad vůbec nepohybují.

1.1 Cíle aplikace

Cílem aplikace „Digitalizace evidence pracovní doby“ je plně automatizovat zpracování a evidenci odpracovaných hodin vyučujícího na vysoké škole. Aplikace importuje rozvrh z IS STAG, umožňuje definovat globální parametry pracovního dne (od–do, obědové pauzy, semestrální nastavení), zobrazuje denní, týdenní či měsíční kalendář, upozorňuje na kolize a překročení týdenního fondu a generuje finální PDF dokument připravený k odevzdání.

1.2 Funkcionalita aplikace

1.2.1 Import rozvrhu ze STAG

Aplikace načítá rozvrh vyučujícího ve formátu CSV exportovaném ze systému STAG. Data jsou parsována do interního modelu, ve kterém jsou zaznamenány typy výuky, časy konání a předmětové kódy.

1.2.2 Globální nastavení pracovních parametrů

Uživatel v sekci „Globální nastavení“ určuje standardní pracovní dobu (např. 8:00–16:30), polední přestávku a režim zimního či letního semestru. Parametry lze aplikovat na všechny dny nebo upravit individuálně pro každý den v týdnu.

1.2.3 Grafické rozhraní pro úpravu denních údajů

V hlavním kalendáři (denním/týdenním/měsíčním režimu) lze kliknutím otevřít formulář „Nová událost“, kde uživatel přidá, upraví či odstraní bloky práce, obědové pauzy nebo jiné události. Překročení týdenního fondu či kolize s rozvrhem jsou vizuálně zvýrazněny.

1.2.4 Automatické značení svátků a volných dnů

Vestavěný kalendář automaticky označí státní svátky a školní prázdniny, které se do výpočtu fondu nepromítají. Tyto dny jsou v tabulce zbarveny odlišně a v celkovém měsíčním součtu se nezapočítávají.

1.2.5 Generování PDF dokumentu

Po kontrole a případné úpravě dat uživatel klepne na „Generovat EPD“. Aplikace vytvoří PDF s hlavičkou (katedra, rok, jméno vyučujícího), detailní tabulkou denních údajů (začátek, pauzy, konec, odpracováno, přesčas, neodpracováno) a součtovým řádkem. PDF je připraveno ke stažení.

1.3 Programovací jazyky

1.3.1 Jazyk C#

Aplikace je napsána v jazyce C#, moderním vysokoúrovňovém objektově orientovaném jazyce vyvinutém společností Microsoft. C# kombinuje vlastnosti jazyků C++ a Java, což mu poskytuje silné typování, automatickou správu paměti prostřednictvím garbage collectoru a bohaté konstrukty pro práci s objekty a kolekcemi.

Kód je kompilován do mezikódu MSIL (Microsoft Intermediate Language), nezávislého na platformě, který je následně uložen do assembly souborů. Při spuštění CLR (Common Language Runtime) provede JIT (Just-In-Time) kompilaci, která převede MSIL do nativního strojového kódu přesně tak, aby odpovídal cílovému operačnímu systému a procesoru. Tento dvoufázový proces zaručuje jak vysokou přenositelnost kódu mezi Windows a Linuxem, tak optimalizaci výkonu pro konkrétní hardware.

Platforma .NET 8.0 navíc poskytuje rozsáhlou standardní knihovnu (BCL – Base Class Library), správu závislostí přes NuGet a vestavěné bezpečnostní mechanismy, které usnadňují práci s I/O, vlákny a asynchronním programováním. C# podporuje nejen objektově orientované programování (OOP), ale i komponentově-orientované, funkcionální i reaktivní paradigmata, což vývojářům umožňuje psát čistý, modulární a testovatelný kód.

Pravidelné aktualizace .NET a C# přinášejí nové syntaktické zkratky, vylepšené výkonové optimalizace i rozšíření frameworku, díky čemuž lze udržovat krok s moderními trendy. V kontextu této aplikace využíváme C# k definici datových modelů, implementaci obchodní logiky (import CSV, výpočty fondu pracovní doby) a vázání na uživatelské rozhraní v Avalonia UI, přičemž .NET zajistí spolehlivé a konzistentní běhové prostředí na obou hlavních desktopových platformách.

1.3.2 Jazyk SQL

SQL (Structured Query Language) je průmyslový standard pro práci s relačními databázemi, vyvinutý na počátku 70. let společností IBM. Díky své deklarativní povaze umožňuje vývojářům a administrátorům databází pohodlně definovat strukturu dat a provádět nad nimi operace bez nutnosti detailního popisu algoritmů zápisu a čtení. Sadu příkazů SQL lze obecně rozdělit na několik kategorií:

- **Definice datového modelu (DDL)** – sem patří příkazy jako CREATE, ALTER a DROP, které slouží k vytváření či úpravě tabulek, indexů, pohledů a dalších databázových objektů.
- **Manipulace s daty (DML)** – do této skupiny spadají příkazy SELECT, INSERT, UPDATE a DELETE, jež umožňují vkládat, vyhledávat, měnit či mazat řádky v tabulkách.
- **Řízení přístupu (DCL)** – pomocí GRANT a REVOKE se spravují uživatelská oprávnění a přístupová práva k různým částem databáze.

- **Transakční příkazy (TCL)** – BEGIN TRANSACTION, COMMIT a ROLLBACK zajišťují, že skupiny operací nad daty proběhnou buď kompletně, nebo se v případě chyby vrátí k původnímu stavu, čímž se zachovává konzistence úložiště.

Pro vykonání SQL dotazu se klient (např. aplikace nebo administrační nástroj) připojí k databázovému serveru pomocí připojovacího řetězce, v němž jsou uvedeny adresa serveru, jméno databáze a přihlašovací údaje. Server následně provede analýzu a optimalizaci dotazu, vybere nejefektivnější plán jeho vykonání, provede operace nad daty a vrátí výsledky zpět klientovi. Tento model odděluje deklaraci požadavků na data (SQL) od jejich fyzického naplnění a zajišťuje přenositelnost kódu mezi různými databázovými systémy.

1.3.4 XAML (.axaml) v Avalonia UI

Avalonia UI využívá deklarativní značkovací jazyk velmi podobný XAML, označovaný často jako .axaml. Tento formát umožňuje vývojářům popsat strukturu a vzhled uživatelského rozhraní odděleně od aplikační logiky. Pomocí .axaml souborů definujete ovládací prvky (tlačítka, vstupní pole, panely), jejich vlastnosti a vztahy, přičemž použité atributy a prvky odpovídají standardům Avalonie.

Například pro vytvoření tlačítka s textem a šířkou zadáte:

```
<Button Content="Uložit" Width="120" Height="40"/>
```

Zde <Button> definuje prvek ovládacího rozhraní, Content určuje popisek, Width a Height jeho velikost.

Při sestavení projektu se .axaml kompilují do binárního formátu v rámci assembly, což zkracuje dobu načítání UI a zajišťuje integraci se systémem stylů (například Fluent Theme). Díky podpoře kompilovaných vazeb (CompiledBindings) jsou kontroly datových bindingů prováděny již v době kompilace, což zvyšuje robustnost aplikace.

XAML (.axaml) v Avalonia UI navíc umožňuje využít interaktivity a chování přes balíčky jako Avalonia.Xaml.Behaviors pro jednoduché animace a logiku bez nutnosti zápisu kódu v code-behindu, čímž se výrazně urychluje vývoj a údržba uživatelského rozhraní.

1.4 Databázové technologie

1.4.1 SQLite

SQLite je lehká vestavěná relační databáze implementovaná v jazyce C, která nevyžaduje samostatný server ani složité nastavení. Veškerá data jsou uložena v jednom souboru na disku, což usnadňuje přenositelnost i zálohování. Autorem SQLite je Richard Hipp a první verze spatřila světlo světa v roce 2000. Díky malému paměťovému i diskovému otisku a optimalizovanému kódu C dosahuje SQLite vysoké rychlosti při běžných SQL operacích. Podporuje plnou transakční integritu (ACID), závazkuje data v okamžiku zápisu a umožňuje paralelní čtení bez zamykání.

SQLite nachází uplatnění v desktopových aplikacích, mobilních zařízeních (Android, iOS), vestavěných systémech i prohlížečích. Je součástí mnoha frameworků a knihoven, což z něj činí oblíbenou volbu pro lokální uložení konfigurací, meziproduktů či celých datových sad bez nutnosti externího serveru.

1.5 Vývojové prostředí

1.5.1 Microsoft Visual Studio

Microsoft Visual Studio patří mezi přední integrovaná vývojová prostředí (IDE), která umožňují vývojářům pokrýt celý životní cyklus softwaru – od psaní a ladění kódu až po nasazení a správu verzí. Díky podpoře řady programovacích jazyků (C#, C++, Visual Basic, F#, Python a dalších) lze v jednom řešení snadno kombinovat různé technologie. Visual Studio poskytuje funkce jako IntelliSense pro inteligentní dokončování kódu, refaktoring pro rychlou úpravu struktury projektů, integrovaný debugger s možností krokování napříč vlákny a profilování výkonu aplikace.

IDE je navrženo tak, aby plynule spolupracovalo s distribuovanými systémy správy verzí – od Git a GitHub přes Azure DevOps až po Subversion nebo Mercurial. Umožňuje vizuální sledování změn, práci s větvemi a slučování kódu přímo v prostředí, bez nutnosti samostatných nástrojů. Rozšíření a doplňky (extensions) navíc umožňují připojit nástroje pro Docker, Kubernetes, testování jednotek, analýzu kvality kódu či integraci s cloudovými službami Azure, AWS nebo Google Cloud.

Visual Studio dále zahrnuje návrhář UI pro WPF, WinForms i webové technologie, integrované editory pro SQL, JSON či XML a správce balíčků NuGet pro snadné přidávání třetích stran. Funkce Live Share navíc umožňuje vzdálenou kolaboraci v reálném čase, což vede k efektivnější týmové práci a rychlejšímu řešení problémů.

1.5.2 SQLiteStudio

SQLiteStudio je multiplatformní grafické rozhraní určené ke správě SQLite databází, které nevyžaduje instalaci serveru a pracuje přímo se souborem databáze. Jeho přehledné GUI dovoluje rychle vytvářet, měnit či mazat tabulky, indexy a pohledy, prohlížet i upravovat záznamy a zároveň exportovat výsledky dotazů do formátů CSV, SQL či HTML.

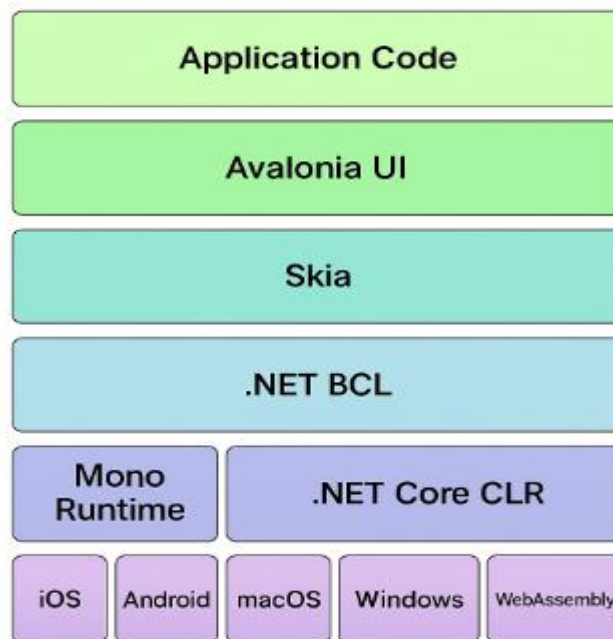
Vestavěný SQL editor podporuje syntaktické zvýraznění, automatické doplňování a historii dotazů, což usnadňuje psaní i ladění složitých SQL příkazů. SQLiteStudio umí zobrazit datový model v podobě ER diagramu, spravovat trigger funkce a definovat cizí klíče. Součástí nástroje je také vizuální návrhář schématu a debugger pro procedurální rozšíření SQLite.

Pro zálohování a obnovu databáze stačí jediný klik, nástroj navíc umožňuje hromadnou úpravu dat, filtrování či seskupování přímo v tabulce. Díky podpoře importu dat z externích zdrojů (CSV, SQL dump) a integrovanému reportingu je SQLiteStudio oblíbenou volbou pro rychlý vývoj prototypů, správu lokálních databází i pro nasazení ve vestavěných systémech.

1.6 Frameworky

1.6.1 Avalonia UI

Avalonia UI je otevřený multiplatformní framework pro tvorbu desktopových aplikací, jehož první verze byla vydána v roce 2016 a aktuální stabilní řada je 11.2.4. Avalonia cílí primárně na desktopové systémy (Windows, Linux, macOS), ale díky podpoře .NET 8.0 může fungovat i v kontejnerizovaných nebo serverových scénářích.



Obrázek 1: Architektura aplikace v .NET

Avalonia používá deklarativní značkovací jazyk .axaml, který se podobá XAML z WPF, a kompiluje jej do optimalizované podoby při sestavení aplikace. Vrstvu vykreslování zajišťuje vlastní multiplatformní renderer, který využívá Direct2D na Windows, Skia na Linuxu a macOS a OpenGL pro scénáře, kde nejsou nativní API dostupná. Tím je dosažena vysoká konzistence vzhledu i chování napříč platformami.

Architektura Avalonia UI vychází z návrhového vzoru MVVM. Pohledy (View) jsou definovány v .axaml souborech, viewmodely (ViewModel) v C# třídách a modelová vrstva (Model) reprezentuje data a obchodní logiku. Pro kompilované vazby (CompiledBindings) se v projektu nastavuje `AvaloniaUseCompiledBindingsByDefault`, což umožňuje odhalit chyby v bindingech již v době sestavení.

Styling a témata jsou řešena prostřednictvím balíčků jako `Avalonia.Themes.Fluent`, které poskytují moderní Fluent vzhled, a `Avalonia.Fonts.Inter` pro čitelné písmo. K interaktivitě a animacím slouží `Avalonia.Xaml.Interactivity` a `Avalonia.Xaml.Behaviors`.

Při spuštění aplikace Avalonia načte sestavené axaml zdroje, inicializuje renderer a vytvoří strom vizuálních prvků, které reagují na systémové události a uživatelské vstupy. Díky Dependency Injection (např. pomocí Splat nebo `Microsoft.Extensions.DependencyInjection`) lze snadno registrovat služby, viewmodely i komponenty pro preview PDF nebo správu dat.

Integrace s ekosystémem .NET 8.0 a NuGet balíčky umožňuje rozšířit Avalonia projekt o další funkce – od reaktivního programování s `ReactiveUI`, přes generování PDF s `QuestPDF` či `PdfSharpCore` až po získávání svátků s `Nager.Date`. Tím vývojáři získávají jednotné prostředí pro rychlou tvorbu robustních, stylově jednotných a snadno udržitelných desktopových aplikací.

1.7 Architektura desktopové aplikace

Architektura desktopové aplikace určuje, jak jsou rozčleněny její hlavní části, jak spolu komunikují a jak je zajištěna udržitelnost, rozšiřitelnost a výkon. Nejběžnější přístupy jsou:

- **Vrstvená (Layered) architektura**
 1. **Prezentační vrstva (UI)** – zobrazuje data a přijímá uživatelské akce.
 2. **Aplikační vrstva (ViewModel/Controller)** – realizuje obchodní logiku a zprostředkovává tok dat mezi UI a modelem.
 3. **Datová vrstva (Model/Repository)** – stará se o přístup k lokálnímu úložišti (např. SQLite), souborům či externím API.
 4. **Sdílené komponenty** – pomocné knihovny, validátory, logging a konfigurace.
- **MVVM (Model-View-ViewModel)**

Zejména u .NET desktopových aplikací (Avalonia, WPF) se často používá MVVM, kde:

 - **Model** obsahuje datové struktury a přístup k datům.
 - **ViewModel** vystavuje data z modelu jako vlastnosti a příkazy pro View.
 - **View** (XAML/.axaml) definuje vizuální rozhraní a váže se na ViewModel pomocí data bindingu.
- **Plugin-based architektura**

Aplikace může být navržena tak, že jádro poskytuje základní funkce, zatímco rozšiřující moduly (pluginy) se mohou dynamicky přidávat či odebírat. To podporuje modulární rozšíření bez zásahu do jádra.
- **Event-driven prvky**

Pro interní komunikaci komponent se využívají události (events) a zprávové toky. Díky Event Aggregatoru či Mediatoru lze udržet volnou vazbu mezi modulárními částmi aplikace.

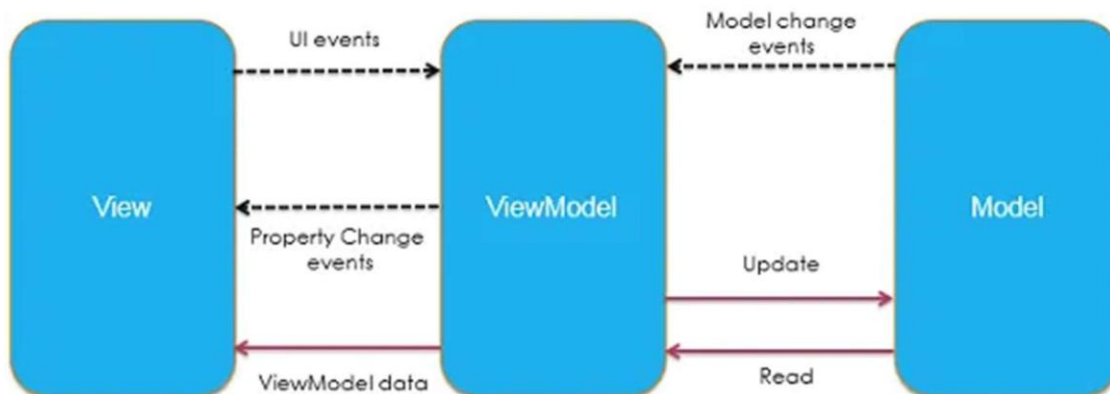
Klíčové principy kvalitní desktopové architektury:

- **Separation of Concerns (SoC)** – každá část (UI, logika, data) je jasně oddělena.
- **Dependency Injection** – umožňuje vkládání závislostí z venku, což usnadňuje testování a výměnu komponent.
- **Modularita a znovupoužitelnost** – vyvíjené moduly lze znovu používat v jiných projektech.
- **Testovatelnost** – čisté rozdělení vrstev a použití MVVM umožňuje psát jednotkové testy bez UI.
- **Udržitelnost** – díky dobře definovaným konvencím a vrstvám je změna nebo rozšíření funkce přehledné.

- **Výkon a odezva** – asynchronní operace I/O a threading (Task, async/await) zajišťují, že UI zůstane plynulé.

1.7.1 Architektura MVVM

Architektura Model-View-ViewModel (MVVM) je návrhový vzor určený k oddělení uživatelského rozhraní od obchodní logiky a modelu dat. Na schématu níže (Obrázek 2) jsou znázorněny tři samostatné vrstvy:



Obrázek 2: Architektura MVVM

- **Model**
Vrstva Model představuje doménová data a všechny operace s nimi spojené (přístup do databáze, výpočty, validace). Model neobsahuje informace o tom, jak jsou data zobrazována – stará se pouze o jejich správu a uchování.
- **View (Zobrazení)**
View definuje vizuální podobu aplikace a zachycuje vstupy od uživatele (kliknutí, zadávání textu). Měla by být co nejtenčí, bez vlastní logiky pro zpracování dat. Často se realizuje pomocí XAML nebo jiného deklarativního jazyka.
- **ViewModel**
ViewModel funguje jako prostředník mezi Modelem a Zobrazením. Exponuje data z Modelu jako vlastnosti a události (např. ObservableCollection), poskytuje příkazy (ICommand) k obsluze akcí uživatele a převádí požadavky z View na volání metod v Modelu. ViewModel může obsahovat logiku týkající se formátování nebo stavu prezentace.

Díky čistému oddělení těchto tří součástí přináší MVVM řadu výhod:

- **Testovatelnost** — ViewModel je testovatelný nezávisle na UI.
- **Modularita** — změny v UI neovlivní Model ani logiku aplikace.
- **Opakované použití** — jeden ViewModel lze navázat na více typů zobrazení.
- **Týmová spolupráce** — vývojáři a návrháři UI mohou pracovat paralelně.

1.8 Knihovny

V této kapitole podrobně popíšeme hlavní NuGet balíčky využívané v aplikaci, jejich účel a klíčové vlastnosti.

1.8.1 Knihovna Avalonia

Avalonia je jádro multiplatformního UI frameworku, které poskytuje základní API pro vytváření oken, layoutů a ovládacích prvků. Díky deklarativnímu jazyku .axaml a kompilovaným vazbám (CompiledBindings) umožňuje čisté oddělení prezentační vrstvy od aplikační logiky a garantuje konzistentní chování na Windows, Linuxu i macOS.

1.8.2 Knihovna Avalonia.Desktop & Avalonia.Diagnostics

Avalonia.Desktop rozšiřuje základní balíček o optimalizace a závislosti potřebné pro desktopové operace (např. native windowing).

Avalonia.Diagnostics přidává vizuální inspektor a nástroje pro ladění UI – umožňuje analyzovat vizuální strom, bindingy a výkonnostní metriky přímo za běhu aplikace.

1.8.3 Knihovna Avalonia.ReactiveUI

Avalonia.ReactiveUI zajišťuje hladkou integraci Avalonia UI s reaktivním programováním pomocí ReactiveUI. Podporuje deklarativní příkazy a bindování IObservable proudů, což usnadňuje implementaci MVVM vzoru.

1.8.4 Knihovna Avalonia.Themes.Fluent & Avalonia.Fonts.Inter

Avalonia.Themes.Fluent poskytuje moderní Fluent vzhled ovládacích prvků a podporu dynamických témat.

Avalonia.Fonts.Inter zpřístupňuje font Inter, optimalizovaný pro čitelnost na obrazovkách, a usnadňuje jednotné typografické řešení v celé aplikaci.

1.8.5 Knihovna Avalonia.Xaml.Behaviors & Avalonia.Xaml.Interactivity

Tyto knihovny umožňují přidávat k ovládacím prvkům chování a jednoduché animace přímo v .axaml, aniž by bylo nutné zasahovat do code-behind. Podporují například drag-and-drop, detekci událostí či spouštění příkazů na základě uživatelských akcí.

1.8.6 Knihovna ReactiveUI

ReactiveUI je nezávislý framework pro reaktivní MVVM, který přidává do ViewModelů podporu pro ObservableAsPropertyHelper, ReactiveCommand a další reaktivní konstrukty. Zajišťuje čisté a testovatelné oddělení prezentační logiky.

Avalonia.Diagnostics přidává vizuální inspektor a nástroje pro ladění UI – umožňuje analyzovat vizuální strom, bindingy a výkonnostní metriky přímo za běhu aplikace.

1.8.7 Knihovna Entity Framework Core & SQLite

Microsoft.EntityFrameworkCore poskytuje ORM vrstvu pro plnou manipulaci s relačními entitami.

Microsoft.EntityFrameworkCore.Sqlite přidává SQLite provider pro lokální souborovou databázi.

Microsoft.EntityFrameworkCore.Design obsahuje nástroje pro generování migrací, analýzu modelu a další design-time funkce.

1.8.8 Knihovna Nager.Date

Nager.Date nabízí jednoduché API pro získání seznamu státních svátků a víkendů pro různá území. V aplikaci se využívá k automatickému značení volných dnů v kalendáři a správnému odpočtu fondu pracovní doby.

1.8.9 Knihovna QuestPDF

QuestPDF poskytuje deklarativní API pro tvorbu komplexních a stylově jednotných PDF šablon. Usnadňuje definování layoutu pomocí flu-like konstrukcí (stack, row, column) a podporuje moderní typografii.

1.8.10 Knihovna Ghostscript.NET

Ghostscript.NET je wrapper pro Ghostscript, který slouží k post-processingu PDF, například ke kompresi, převodu a slučování souborů. V aplikaci se využívá pro finální úpravy a optimalizaci vygenerovaných dokumentů.

1.8.11 Knihovna MessageBox.Avalonia

MessageBox.Avalonia přidává do Avalonia jednoduché dialogové okno pro zobrazování informačních, varovných a chybových hlášek. Umožňuje rychlou implementaci uniformního UX pro komunikaci s uživatelem.

2 PRAKTICKÁ ČÁST

V této kapitole popisujeme realizaci a konstrukci aplikace. Nejprve se věnujeme návrhu SQLite databáze, která uchovává zásadní informace pro chod systému – záznamy uživatelských událostí, denní i globální nastavení uživatele. Databáze je navržena tak, aby byla co nejjednodušší a zároveň výkonná pro potřeby této aplikace.

Dále představujeme organizaci projektových souborů. Struktura řešení je rozdělena do vývojových modulů a složek, jež dohromady tvoří kompletní aplikaci. Každý soubor zahrnuje kód odpovídající konkrétní funkci – od správy dat přes logiku uživatelského rozhraní až po další klíčové komponenty. Díky jasnému členění adresářů se snadno orientuje v jednotlivých částech kódu.

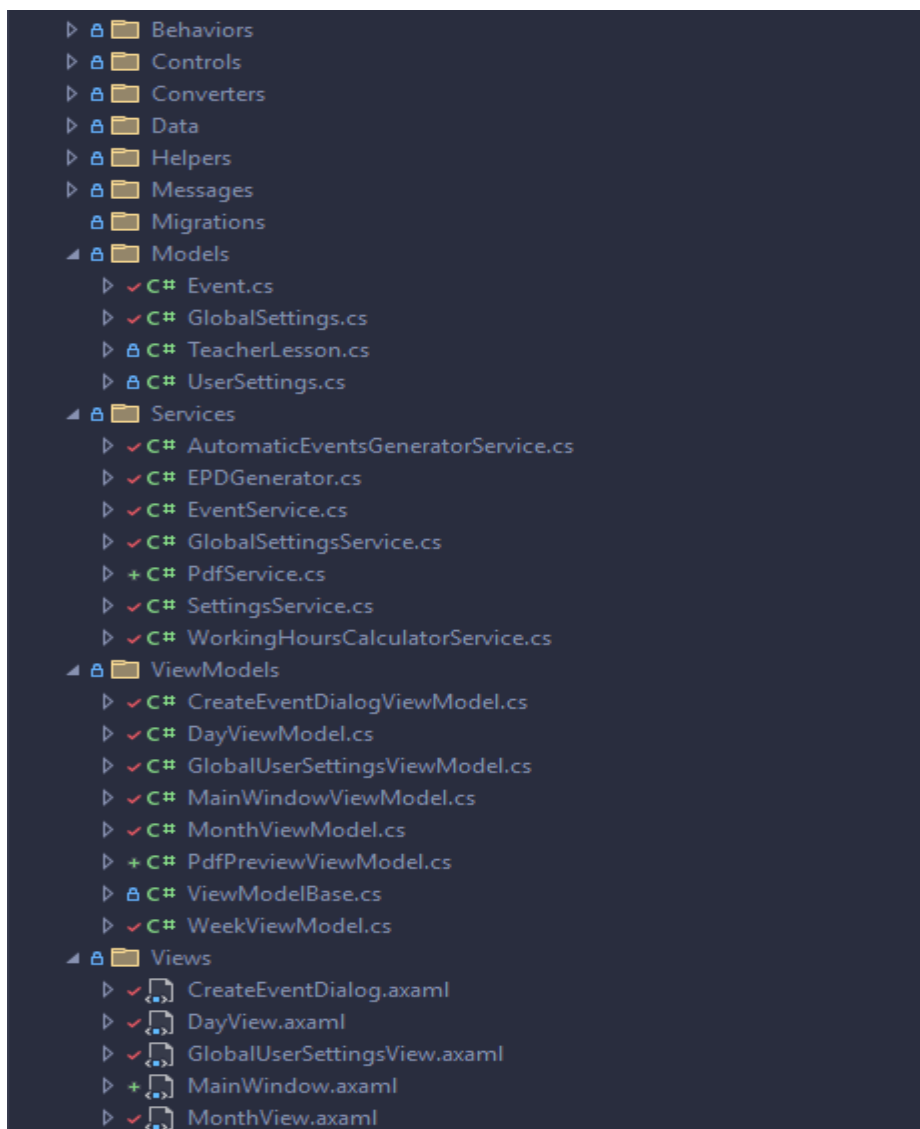
Kapitola rovněž popisuje uživatelské rozhraní, navržené s důrazem na intuitivní ovládání a komplexní přístup k manipulaci s uloženými daty. Rozhraní bylo vytvořeno v jazyce .axaml, který umožňuje bohaté vizuální prvky a jednoduché vazby. Výsledkem je přívětivá aplikace, jež nabízí všechny potřebné nástroje pro efektivní správu událostí a dalších informací.

Shrnuto, kapitola poskytuje ucelený pohled na technickou realizaci aplikace – od databázového modelu přes architekturu souborů až po detaily uživatelského rozhraní.

2.1 Souborová struktura aplikace

Souborová struktura slouží k jasnému oddělení jednotlivých částí aplikace podle jejich zodpovědností a funkcí. Díky logickému členění do samostatných adresářů lze jednoduše najít a upravit kód patřící k datové vrstvě, prezentační vrstvě, či podpůrným službám. Dobře navržená struktura také podporuje princip Separation of Concerns – každá složka obsahuje kód týkající se jednoho tématu (např. modely dat, view-modely, pohledy, služby apod.), což zvyšuje čitelnost, usnadňuje testování a umožňuje paralelní práci více vývojářů.

Kromě toho modulární rozdělení do složek zjednodušuje přidávání nových funkcionalit či úpravu existujících částí bez rizika nežádoucích dopadů na zbytek aplikace. Správně strukturovaný projekt usnadňuje i nasazení a automatizaci build-procesu (CI/CD), protože jednotlivé moduly lze kompilovat, testovat a nasazovat odděleně. Celkově tak přehledná souborová struktura tvoří základ pro efektivní vývoj, údržbu a další rozvoj desktopové aplikace.



Obrázek 3: Souborová struktura aplikace. 1

Souborová struktura aplikace je rozdělena do následujících hlavních složek:

- **Behaviors**
- **Controls**
- **Converters**
- **Data**
- **Helpers**
- **Messages**

- **Migrations**
- **Models**
- **Services**
- **ViewModels**
- **Views**

Každá z těchto složek sdružuje soubory související s jednou oblastí funkcionality – např. Data obsahuje definici kontextu databáze a migrace, ViewModels drží MVVM logiku a Views šablony uživatelského rozhraní. V kořenové složce pak najdeme klíčové spouštěcí a konfigurační soubory (App.axaml, app.manifest, Program.cs, ViewLocator.cs).

Toto uspořádání pomáhá udržet kód přehledný, podporuje princip Separation of Concerns a usnadňuje paralelní vývoj, testování i nasazení jednotlivých modulů aplikace.

2.1.1 Složka Behaviors

Složka Behaviors obsahuje třídy, které pomocí Avalonia.Xaml.Behaviors přidávají specifické interaktivní chování k UI prvkům, aniž by bylo nutné zasahovat přímo do jejich šablon. Mezi implementovanými režimy jsou například:

- AttachCalendarPanelBehavior – automatické vložení nebo konfigurace vlastního CalendarPanel při načítání okna.
- DayCellClickBehavior, EventClickBehavior, TimeInputBehavior – detekce a předávání událostí kliknutí či vstupu času z jednotlivých částí kalendáře do ViewModelu.
- CloseDialogBehavior, ConfirmDeleteBehavior – zajištění standardizovaného zavírání dialogů a jednorázového potvrzení akce s možností zrušení.
- ShowValidationMessageBehavior, ShowCollisionMessageBehavior – zobrazení chybových či varovných hlášení (např. kolize událostí) ve formě Tooltipů nebo pop-up zpráv.

Behaviors tak umožňují udržet čistý oddělený kód vizuální logiky a snadno přidávat či měnit interakce bez úprav základního XAML.

2.1.2 Složka Controls

Složka Controls obsahuje vlastní uživatelské ovládací prvky, které rozšiřují základní komponenty Avalonia UI. Jde o znovupoužitelné elementy (např. speciální panely či kontejnery pro zobrazení událostí), jež kombinují prezentaci i část logiky rozmístování a interakce v rámci aplikačního rozhraní. Tato struktura usnadňuje udržovat čistý a modulární kód UI a rychle zavádět nové vizuální komponenty bez zásahu do ostatních částí aplikace.

2.1.3 Složka Converters

Složka Converters obsahuje třídy implementující převodníky (IValueConverter), které zprostředkovávají konverzi dat mezi ViewModely a View. Tyto konvertory umožňují například:

- přeměnu logických hodnot na viditelnost či barvy (BoolToVisibilityConverter, BoolToBrushConverter),

- transformaci časových údajů a posunů (DateTimeToDateTimeOffsetConverter, HourToMarginConverter, HourToOffsetConverter, HoursToHeightConverter),
- výpočet pozic a šířek pro vykreslení událostí v kalendáři (DayToCanvasLeftConverter, DayToColumnWidthConverter),
- převod výčtových typů na boolean pro dvěcestné vazby (EnumToBooleanConverter),
- a určení barvy prvků na základě typu události (EventBrushConverter).

Díky těmto konvertorům je možné udržet čistý MVVM vzor – logika transformace dat zůstává mimo pohled (View) a je centralizovaná, což zvyšuje přehlednost, usnadňuje údržbu a podporuje opakované použití převodních tříd napříč celou aplikací.

2.1.4 Složka Data

Složka Data obsahuje jediný klíčový soubor AppDbContext.cs (viz obr. 4), který definuje kontext Entity Framework Core pro přístup k SQLite databázi.

Třída AppDbContext

- Deklaruje sady (DbSet <>) pro tabulky Events, UserSettings a GlobalSettings.
- V metodě OnConfiguring detekuje operační systém a vytváří složku TeacherScheduleApp ve výchozím adresáři uživatelských dat (AppData na Windows, ~/.config na ostatních platformách). Poté nakonfiguruje připojení k SQLite souboru v tomto umístění.
- V OnModelCreating registruje převodník (ValueConverter) pro vlastnosti StartTime a EndTime v entitě Event, který zajišťuje ukládání a načítání datum-časových hodnot ve formátu yyyy-MM-dd HH:mm:ss.

Tím AppDbContext zajišťuje plně asynchronní a typově bezpečné čtení i zápis do lokální databáze, včetně automatického vytvoření datového souboru a aplikace všech definovaných konvencí a převodů.

```

public class AppDbContext : DbContext
{
    Ссылка: 14 | Egor Razboinikov, 27 дн. назад | Автор: 1, изменение: 1
    public DbSet<Event> Events { get; set; }
    Ссылка: 5 | Egor Razboinikov, 27 дн. назад | Автор: 1, изменение: 1
    public DbSet<UserSettings> UserSettings { get; set; }
    Ссылка: 5 | Egor Razboinikov, 13 дн. назад | Автор: 1, изменение: 1
    public DbSet<GlobalSettings> GlobalSettings { get; set; }

    Ссылка: 0 | Egor Razboinikov, 27 дн. назад | Автор: 1, изменение: 1
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        string baseDir;
        if (RuntimeInformation.IsOSPlatform(OSPlatform.Windows))
            baseDir = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData);
        else
            baseDir = Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.UserProfile), ".config");

        var appDir = Path.Combine(baseDir, "TeacherScheduleApp");
        Directory.CreateDirectory(appDir);

        var dbPath = Path.Combine(appDir, "teacherapp.db");
        optionsBuilder.UseSqlite($"Data Source={dbPath}");
    }

    Ссылка: 0 | Egor Razboinikov, 27 дн. назад | Автор: 1, изменение: 1
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        var dateTimeConverter = new ValueConverter<DateTime, string>(
            v => v.ToString("yyyy-MM-dd HH:mm:ss", CultureInfo.InvariantCulture),
            v => DateTime.ParseExact(v, "yyyy-MM-dd HH:mm:ss", CultureInfo.InvariantCulture)
        );
        modelBuilder.Entity<Event>()
            .Property(e => e.StartTime)
            .HasConversion(dateTimeConverter);

        modelBuilder.Entity<Event>()
            .Property(e => e.EndTime)
            .HasConversion(dateTimeConverter);
        base.OnModelCreating(modelBuilder);
    }
}

```

Образек 4: Ukázka kódu třídy AppDbContext 1

2.1.5 Složka Helpers

Složka Helpers obsahuje obecné pomocné třídy a utility, které podporují různé části aplikace drobnými, opakovaně použitelnými funkcemi. V našem projektu najdeme například:

- Helper.cs – poskytuje metodu GetMainWindow (), která vrátí odkaz na hlavní okno aplikace (MainWindow) z kontextu Avalonia desktopového životního cyklu. Díky tomu mohou ostatní části kódu snadno získat přístup k instanci hlavního okna, aniž by bylo nutné předávat referenci explicitně.
- HolidayHelper.cs – komponenta, která využívá knihovnu Nager.Date pro získání seznamů státních svátků a dále zpracovává data tak, aby byla ready-to-use v logice generátoru událostí.

Díky umístění těchto utilit do samostatné složky Helpers zůstávají ostatní moduly čisté a mohou se zaměřit na svou hlavní odpovědnost, zatímco veškeré opakované a doplňkové funkce jsou dostupné centralizovaně

2.1.6 Složka Messages

Složka Messages obsahuje definice zpráv (eventů), které slouží k interní komunikaci mezi jednotlivými částmi aplikace, zejména mezi ViewModely. Každá třída představuje konkrétní signalizaci změny nebo události:

- `AutoEventsGeneratedMessage` – oznámení, že algoritmus automaticky vygeneroval nové události do kalendáře.
- `EpdGeneratedMessage` – zpráva o dokončení generování PDF dokumentu (Evidence pracovní doby).
- `GlobalSettingsChangedMessage` – notifikace o změně globálního nastavení pracovního režimu.
- `UserSettingsChangedMessage` – upozornění, že se upravilo uživatelské nastavení pro konkrétní den.

Pomocí těchto zpráv a zprostředkovatele (např. `MessageBus`) mohou ViewModely reagovat na změny stavu bez přímých závislostí na sobě navzájem, čímž se udržuje volná vazba a lepší škálovatelnost architektury.

2.1.7 Složka Models

Složka Models obsahuje třídy, které představují datové entity aplikace a jejich strukturu v relační databázi. Jedná se o čisté POCO objekty, jež Entity Framework Core mapuje na odpovídající tabulky v SQLite. Typickými příklady jsou:

- `Event` – popis události v kalendáři (čas začátku, konec, typ, popis).
- `GlobalSettings` – nastavení platné globálně pro celou aplikaci (pracovní hodiny, semestrální režim).
- `UserSettings` – individuální konfigurace uživatele pro konkrétní dny.
- `TeacherLesson` – informace o konkrétním bloku výuky importovaném z rozvrhu.

Tento oddíl kódu definuje, jaká data se ukládají a jaká pole jednotlivé tabulky obsahují, a slouží jako most mezi databází a aplikační logikou.

2.1.8 Složka Models

Složka Services shromažďuje třídy, které zajišťují zásadní funkce aplikace na úrovni business logiky a datové manipulace. Jednotlivé služby se starají o:

- `EventService` – CRUD operace nad entitou `Event`, import/export bloků, a také správa všech událostí včetně automaticky generovaných položek.
- `AutomaticEventsGeneratorService` – vytváří výchozí události (práce a pauzy) na základě staženého rozvrhu a globálních nastavení.
- `WorkingHoursCalculatorService` – počítá odpracovaný fond hodin, přesčasy a nedodělky pro jednotlivé dny.
- `SettingsService` – načítá a ukládá uživatelská nastavení pro daný den.
- `GlobalSettingsService` – spravuje globální nastavení (pracovní hodiny, semestry).
- `PdfService` – poskytuje šablony a výchozí hodnoty (příchody, odchody, pauzy) pro generování PDF dokumentu.

Zvláštní pozornost je třeba věnovat Algoritmus vyvážení, který se nachází v – EventService. K vyrovnaní denních hodinových výkyvů (přebytek / nedostatek oproti očekávanému pracovnímu fondu) slouží metoda BalanceEventsForMonth. Spouští se automaticky při převzetí dat pro náhled PDF a funguje takto:

1. Načtení událostí
Načtou se všechny události daného měsíce z databáze (kromě smazaných).
2. Skupinování podle dnů
Události se rozdělí do skupin podle dne v měsíci, přičemž víkendy a svátky se přeskočí (ostatní dny do seznamu statistik).
3. Statistiky pro každý den
Pro každý den se vytvoří DayStat, který obsahuje:
 - seznam událostí
 - skutečně odpracované hodiny (součet bloků práce a služebních cest)
 - příznak IsFixed, pokud den obsahuje speciální (ne-pracovní) událost pokrývající celý pracovní den
4. Výchozí hodnoty
Pro každý den se načtou buď uživatelská, nebo globální výchozí časy příchodu, odchodu a pauzy, aby bylo jasné, kolik hodin by měl den obsahovat.
5. Výpočet rozdílů
Pro nekrytá (IsFixed = false) statistická data se spočítá Amount = skutečně odpracováno – očekávaný fond (v hodinách).
Dny s kladným Amount mají přebytek, se záporným nedostatek.
6. Vyvážení mezi dny
Nedostatek hodin (deficity) se postupně vykrývá z přebytků z jiných dnů:
 - Pro každý deficit vybíráme dny s přebytkem.
 - Přesouváme časový úsek (delta = min (přebytek, potřeba)) z konce poslední pracovní události dne s přebytkem do konce poslední pracovního bloku dne s nedostatkem.
 - Upravené události se ihned aktualizují v paměti (EndTime -= delta / EndTime += delta) a jejich nová délka se započítá.
7. Uložení změn
Po balančním průchodu se v cyklu:
 - pro každý den s novými hodnotami příchodu/odchodu/pauzy volá SettingsService.SaveUserSettingsForDate
 - pro všechny upravené události se volá UpdateEvent, čímž se změny zapisou do databáze

Díky tomuto algoritmu zůstává týdenní i měsíční fond pracovní doby přesně vyvážený podle nastavených pravidel, a uživatel při náhledu PDF rovnou vidí korigovaná data bez manuálních úprav.

2.1.9 Složka ViewModels

Složka ViewModels obsahuje třídy, které realizují prezentační logiku podle vzoru MVVM. Každý ViewModel zprostředkovává data mezi modelem a odpovídajícím pohledem, vystavuje vlastnosti a příkazy pro interakci s uživatelem a reaguje na změny v datech či nastaveních. Hlavní soubory ve složce jsou:

- ViewModelBase.cs – základní třída poskytující implementaci INotifyPropertyChanged a sdílené služby pro všechny ViewModely.
- MainWindowViewModel.cs – orchestruje načítání a přepínání mezi jednotlivými zobrazeními (den, týden, měsíc) a spouští hlavní příkazy aplikace.
- DayViewModel.cs, WeekViewModel.cs, MonthViewModel.cs – spravují data a logiku pro zobrazení kalendáře v denním, týdenním a měsíčním režimu.
- CreateEventDialogViewModel.cs – řídí dialog pro vytváření či editaci události, zpracovává vstup uživatele a validuje údaje.
- PdfPreviewViewModel.cs – připravuje data pro náhled PDF a spouští generování vyrovnaného dokumentu.
- GlobalUserSettingsViewModel.cs – umožňuje úpravy globálních a uživatelských nastavení, která ovlivňují výpočet fondu pracovní doby.

Díky tomuto oddělení zůstává prezentační logika soustředěna ve ViewModel vrstvách, Views jsou čistě deklarativní a jednotlivé moduly lze snadno testovat či měnit nezávisle na sobě.

2.1.10 Složka Views

Složka Views obsahuje šablony uživatelského rozhraní ve formě .axaml souborů a jejich code-behind třídy, které definují vizuální vrstvy aplikace. Každý View je navržen pro konkrétní část funkcionality:

- MainWindow.axaml – hlavní okno aplikace s navigačním menu a zónou pro zobrazení kalendáře.
- DayView.axaml, WeekView.axaml, MonthView.axaml – XAML šablony pro denní, týdenní a měsíční kalendářní pohled.
- CreateEventDialog.axaml – dialogové okno pro vytváření či úpravu událostí.
- GlobalUserSettingsView.axaml – rozhraní pro úpravu globálních a denních uživatelských nastavení.
- PdfPreviewWindow.axaml – samostatné okno pro náhled vygenerovaného PDF dokumentu.

Díky tomuto rozdělení zůstává prezentační vrstva čistě deklarativní, usnadňuje se práce na vzhledu aplikace a Views lze rychle upravovat bez ovlivnění obchodní logiky.

2.1.11 Složka Kořenové soubory projektu

App.axaml je deklarativní XAML soubor, který definuje globální zdroje aplikace — například styly (témata), šablony datových šablon (DataTemplate) a ostatní sdílené zdroje (barvy, fonty). V horní úrovni se obvykle registrují i konvertory a globální ResourceDictionary. Tento soubor zajišťuje jednotný vzhled a chování celého UI a načítá se při spuštění aplikace.

App.axaml.cs kódový-behind k App.axaml obsahuje třídu App : Application, kde se v metodě OnFrameworkInitializationCompleted() provádí inicializace hlavního okna (MainWindow), konfigurace life-cycle (např. nastavení IClassicDesktopStyleApplicationLifetime) a případné spuštění prvotních akcí (např. načtení výchozích dat). Zde se váže deklarativní obsah k reálným objektům a spouští se první obrazovka.

Manifestový soubor app.manifest pro Windows určuje metadata aplikace: identitu (název, ikony), požadovaná oprávnění (např. přístup k sítím nebo souborům), DPI-awareness a další parametry běhového prostředí. Díky manifestu Windows ví, jak aplikaci izolovat a jaké zdroje jí povolit.

Program.cs je vstupním bodem aplikace. Obsahuje statickou metodu Main(), která běžně volá BuildAvaloniaApp().StartWithClassicDesktopLifetime(args). V BuildAvaloniaApp() se konfiguruje AppBuilder — registrují se platformové specifické backendy, načítají se styly (např. FluentDark), zapíná se diagnóza, případně se injektují závislosti (DI container). Celá struktura zde definuje, jakým způsobem se aplikace spustí a jaké moduly se načtou jako první.

ViewLocator implementuje jednoduchou logiku pro automatické mapování ViewModelů na odpovídající Views. Obvykle obsahuje metodu Register <TViewModel, TView>() nebo reflexní pravidlo (např. hledání MainWindow pro MainWindowViewModel). Pomocí DataTemplate v App.axaml pak Avalonia přiřazuje správný View k instanci ViewModelu. Díky tomu lze ve ViewModelech pracovat nezávisle na konkrétních UI prvcích a navazovat navigaci čistě na typovou úroveň.

Těmito soubory je definována a inicializována globální infrastruktura aplikace — od zdrojů a vzhledu (App.axaml), přes startovací logiku (App.axaml.cs, Program.cs), až po dynamické mapování prezentační vrstvy (ViewLocator.cs) a základní metadata (app.manifest).

2.2 Struktura databáze

V dnešní době se bez databáze neobejde žádná aplikace, proto se v této části bakalářské práce zaměříme na její strukturu a návrh. Databáze této aplikace je určena k ukládání různých informací o uživateli. Obsahuje tři tabulky, z nichž každá je zodpovědná za ukládání specifických informací. Tyto tabulky dohromady tvoří páteř databáze a zajišťují správu informací o událostech, nastavení uživatele pro konkrétní den a globální nastavení.

2.2.1 Tabulka Events

Tabulka Events slouží k ukládání všech časových bloků v rozvrhu učitele. Každý řádek má unikátní Id primární klíč a následující sloupce:

- Title – popis události, například název předmětu.
- StartTime, EndTime – čas začátku a konce události ve formátu yyyy-MM-dd HH:mm:ss.
- AllDay – příznak celodenní události (0 nebo 1).
- Description – volný popis či poznámka.
- IsDeleted – logické smazání (0 = aktivní, 1 = vymazáno).
- EventType – číselný kód typu události (práce, oběd, služební cesta apod.).

- ArrivalTime, DepartureTime, LunchStart, LunchEnd – detailní časy dne pro případné manuální úpravy.
- IsAutoGenerated – příznak, že událost byla automaticky vygenerována (0 nebo 1).
- HasCollision – indikátor kolize s jinou událostí (0 nebo 1).
- AutoGeneratedForDate – datum, pro které byla událost vygenerována.

2.2.2 Tabulka GlobalSettings

Tabulka GlobalSettings uchovává výchozí nastavení pracovní doby a parametrů pro celý semestr. Každý záznam má unikátní Id a obsahuje:

- GlobalStartTime, GlobalEndTime – standardní začátek a konec pracovního dne.
- EmployeeName, Department – jméno vyučujícího a název katedry či oddělení.
- Po čtyři sloupce pro každý pracovní den (pondělí – pátek):
 - {Day}Arrival, {Day}Departure – výchozí příchod a odchod.
 - {Day}LunchStart, {Day}LunchEnd – výchozí začátek a konec polední přestávky.
- MinBreakDuration, MaxBreakDuration – minimální a maximální délka povolené přestávky.
- AutoEventNamePreLunch, AutoEventNameLunch, AutoEventNamePostLunch – přednastavené názvy automaticky generovaných bloků (před obědem, oběd, po obědě).
- Semester – identifikátor semestru (např. 0 = zimní, 1 = letní).

2.2.2 Tabulka UserSettings

Tabulka UserSettings ukládá individuální úpravy pracovního dne pro konkrétní datum. Každý záznam má unikátní Id (PK AUTOINCREMENT) a tyto sloupce:

- Date – datum ve formátu yyyy-MM-dd.
- ArrivalTime, DepartureTime, LunchStart, LunchEnd – ručně zadané časy příchodu, odchodu a začátku/konce oběda pro daný den.

Tyto úpravy mají prioritu nad globálními nastaveními z tabulky GlobalSettings a aplikace je použije při generování a vyvažování událostí.

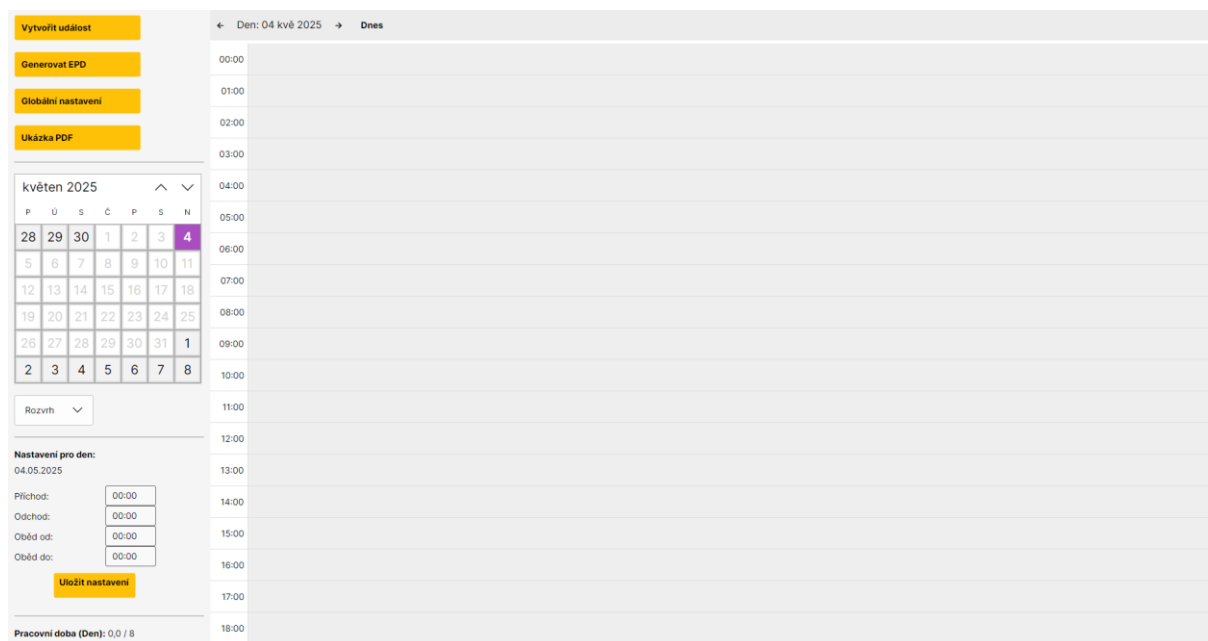
2.3 Uživatelské rozhraní aplikace

Uživatelské rozhraní aplikace je někdy nejdůležitějším aspektem, který určuje, jak snadná a intuitivní bude pro uživatele interakce s aplikací. Jednoduchý a přímočarý návrh uživatelského rozhraní zajišťuje, že aplikace je intuitivní, snadno se používá. To pomáhá uživateli rychle a snadno pochopit, jak aplikace funguje, a může efektivně využívat její funkce. Význam uživatelského rozhraní by se neměl podceňovat. Koneckonců je to tvář aplikace a měla by být atraktivní. Pokud rozhraní obsahuje velké množství prvků nebo je obtížné se v něm orientovat, může to mít negativní dopad na uživatelský zážitek. Naopak dobře navržené uživatelské rozhraní s jasným vizuálem, a nejen strukturou může pro uživatele hrát velmi velkou roli. Klíčovými prvky dobrého uživatelského rozhraní jsou jednoduchost, přehlednost a konzistence.

2.3.1 Hlavní stránka aplikace

První věc, kterou uživatel uvidí po spuštění aplikace, je její hlavní stránka. Při prvním spuštění aplikace se také automaticky vytvoří události na celý rok. Hlavní stránka je rozdělena na dva hlavní panely. Levý panel slouží k různým interakcím s aplikací. Druhý panel slouží k zobrazení pohledů na daný den, týden nebo měsíc v závislosti na tom, co uživatel vybral v rozevíracím seznamu na levém panelu. Výchozí zobrazení je aktuální den.

Popíšeme podrobně levý panel. V horní části tohoto panelu jsou čtyři tlačítka, která slouží k vyvolání různých funkcí. První tlačítko „Vytvořit událost“ slouží k vyvolání dialogového okna (jeho popis je uveden níže). Druhé tlačítko „Generovat EPD“ slouží k vyvolání dialogového okna pro načtení souboru .csv s rozvrhem učitele ze STAG. Po výběru souboru se spustí proces vytváření událostí ze souboru. Třetí tlačítko „Globální nastavení“ slouží k otevření okna globálních nastavení (podrobný popis níže). Čtvrté tlačítko „Příklad PDF“ slouží k otevření dialogového okna pro prohlížení dokumentu ve formátu PDF. Za tlačítky se nachází kalendář s možností výběru dne, měsíce a roku. Pokud v tomto kalendáři kliknete na jakýkoli den, aplikace přejde a zobrazí údaje pro tento den. Za kalendářem se nachází rozevírací seznam, který slouží k výběru možnosti zobrazení dne, týdne nebo měsíce. Za seznamem následuje blok nastavení pro konkrétní den. Tam se nacházejí 4 hlavní pole, která slouží k zadání času příchodu, času odchodu a začátku a konce přestávky, a tlačítko, které uloží zadané informace. Stojí za zmínku, že tento blok je viditelný pouze v denním režimu. Za tímto blokem se zobrazují informace o tom, kolik hodin bylo odpracováno z normy.



Obrázek 5: Hlavní stránka aplikace 1

2.3.2 Dialogové okno pro vytvoření události

Dialogové okno obsahuje následující prvky. První pole slouží k zadání nebo úpravě názvu události. Toto pole je povinné, při pokusu o vytvoření události bez názvu se zobrazí dialogové okno s chybou. Druhé pole již není povinné a slouží k přidání popisu události. Poté následuje tlačítko, které určuje, zda bude událost trvat celý den. Poté následuje blok pro nastavení data a času události. Za blokem pro nastavení data je prvek, který umožňuje vybrat typ události (pracovní, volno, služební cesta atd.). A posledními prvky jsou tlačítko pro zrušení, které zavře okno, a tlačítko pro uložení, které událost vytvoří a okno zavře.

Nová událost

Název

Popis

☐ Celý den

Začátek:

4 květen 2025 00:00

Konec:

4 květen 2025 00:00

Typ události: Práce

Zrušit Vytvořit

Obrázek 6: Dialogové okno pro vytvoření 1

2.3.3 Vizuální prezentace pro den

Vizuální prezentace pro daný den obsahuje mřížku s hodinami, pracovní interval je barevně odlišen od nepracovního. V horní části jsou tlačítka, která umožňují přepínat mezi dny. Tam se také nachází tlačítko, které přesune uživatele na aktuální den. Stejně tak zobrazuje události pro daný den, pokud nějaké existují.

← Den: 02 kvě 2025 → Dnes

01:00

02:00

03:00

04:00

05:00

06:00

07:00

08:00

09:00

10:00

11:00

12:00

13:00

14:00

15:00

16:00

17:00

18:00

19:00

Ranní výuka

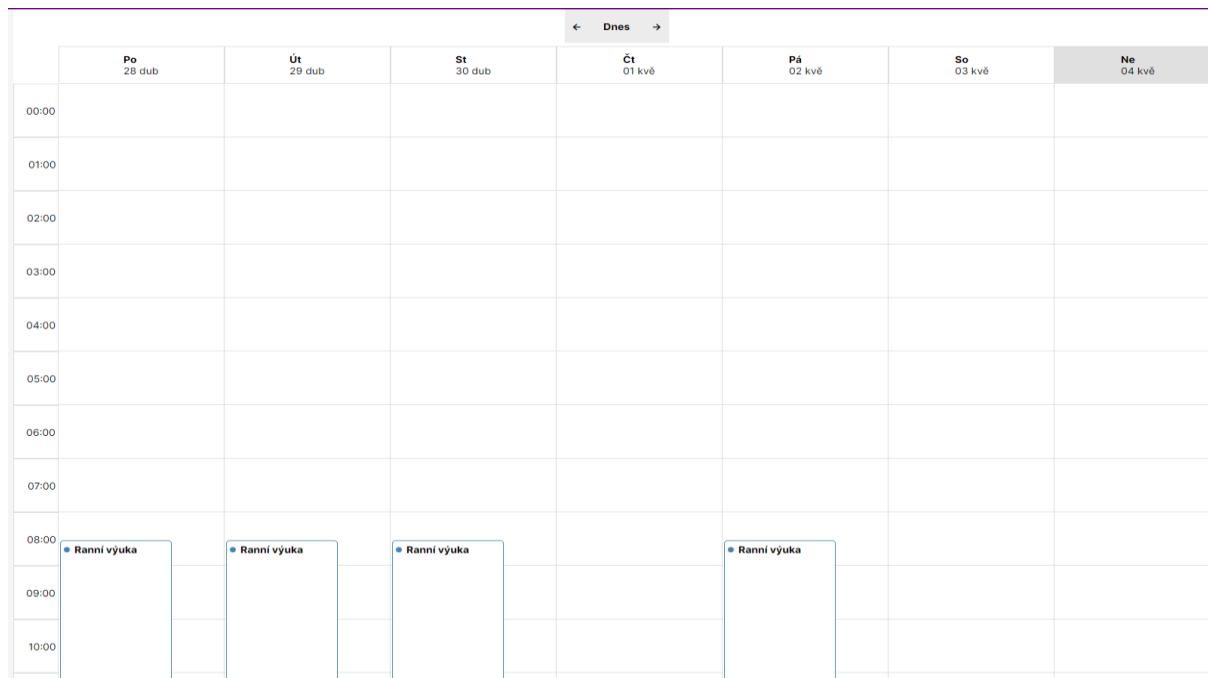
Oběd

Odpolední výuka

Obrázek 7: Vizuální prezentace pro den 1

2.3.3 Vizuální prezentace pro týden

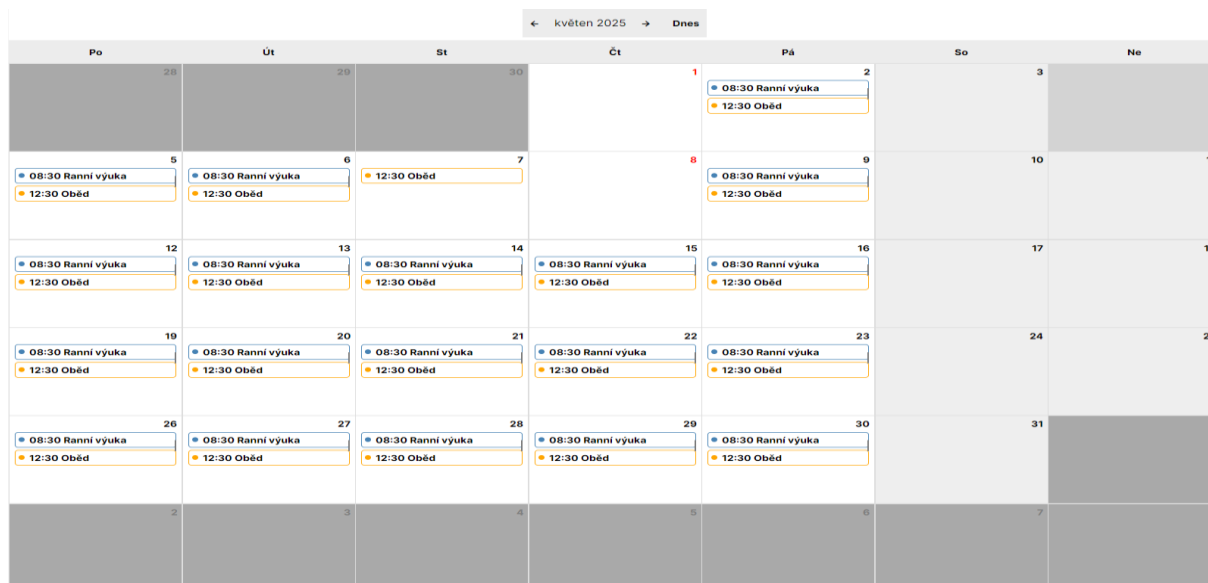
Vizuální prezentace pro týden obsahuje také tabulku s hodinami a dny, které tvoří týden. V horní části jsou tlačítka, která umožňují přepínat mezi týdny. Tam se také nachází tlačítko, které přesune uživatele na aktuální týden. Stejně tak zobrazuje události pro tento týden, pokud nějaké jsou.



Obrázek 8: Vizuální prezentace pro týden 1

2.3.3 Vizuální prezentace pro měsíc

Vizuální prezentace pro měsíc obsahuje také mřížku, ale již s dny, které spadají do tohoto měsíce. Státní svátky, víkendy a dny z předchozího nebo následujícího měsíce se liší barvou pozadí nebo barvou čísla dne. V horní části jsou tlačítka, která umožňují přepínat mezi měsíci. Tam se také nachází tlačítko, které přesune uživatele na aktuální měsíc. Stejně tak zobrazuje události pro tento měsíc, pokud nějaké jsou.



Obrázek 9: Vizuální prezentace pro měsíc 1

2.3.3 Globální nastavení

Globální nastavení obsahuje velké množství polí pro zadání informací o různých časových intervalech a dalších informací o uživateli, které budou použity pro vytváření automatických událostí, generování PDF a tak dále. Podrobný popis začneme tlačítky, která slouží k přepínání mezi zimním a letním semestrem, což uživateli umožňuje zadat různé údaje pro konkrétní semestr. Pole pro zadávání informací jsou stejná pro obě varianty. Dále následuje blok, který odpovídá za globální interval pracovní doby. Poté následuje blok s možností zadání informací o příchodu do práce a odchodu z práce a také času přestávky. Tato pole jsou stejná pro všechny pracovní dny. Záložka další nastavení obsahuje pole pro zadání jména, katedry, které budou použity pro generování v pdf. Pole pro zadání omezení pro přestávku a také jména, která se používají pro vytváření automatických událostí. Na konci se také nachází tlačítko pro uložení.

Globální nastavení uživatele

Zimní semestr

☒ Zimní ☐ Letní

Globální pracovní doba

Od: 08:00 do: 16:30

Zimní semestr

Pondělí Úterý Středa Čtvrtek Pátek Další nastavení

Příchod: 08:00

Odchod: 16:30

Oběd, začátek: 12:00

Oběd, konec: 12:30

Uložit nastavení

Obrázek 10: Globální nastavení 1

2.3.3 Ukázka PDF

Náhled PDF obsahuje dva prvky, se kterými lze interagovat. První slouží k přepínání mezi měsíci, druhý k uložení souboru do počítače. Také obsahuje obrázek, jak bude stažený soubor vypadat.

PDF Preview

2025-05 Save PDF

Evidence pracovní doby, včetně přestávek v práci a práce přesčas

jméno: Radek Matoušek útvar: Katedra informačních technologií pracovní doba: 08:30–17:00 hod. docházka za měsíc: květen 2025

Fond prac. doby: 160 hodin

Den	Začátek pracovní doby	1. přestávka		2. přestávka		Konec pracovní doby	Odpracováno	Přesčas	Neodprac.	Poznámka *)
		Od	Do	Od	Do					
1.										S
2.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
3.										
4.										
5.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
6.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
7.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
8.										S
9.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
10.										
11.										
12.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
13.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
14.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
15.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
16.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
17.										
18.										
19.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
20.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
21.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
22.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
23.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
24.										
25.										
26.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
27.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
28.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
29.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
30.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
31.										
Celkem							160:00:00	00:00:00	00:00:00	

kontr. č. Z odprac. - přesčas + neodprac. hodin (odpovídá FPD): 160:00:00

podpis zaměstnance: _____ podpis nadřízeného pracovníka: _____

*) po poznámky: D – dovolená, N – nemoc, OČR – ošetřování, L – lékař, PC – pracovní cesta, S – svátek a ostatní dny pracovního klidu

Obrázek 11: Ukázka PDF 1

ZÁVĚR

Cílem této bakalářské práce bylo navrhnout, implementovat a ověřit multiplatformní desktopovou aplikaci pro digitalizaci evidence pracovní doby vyučujících, která automatizuje zpracování vstupních dat (rozdělí ze STAG, přestávky, školní prázdniny a státní svátky) a generuje validní PDF dokument připravený k odevzdání. V teoretické části práce jsme analyzovali klíčové technologie a návrhové vzory využité při vývoji aplikace. Popisovali jsme programovací jazyk C#, relační databázi SQLite a deklarativní značkovací jazyk .axaml v Avalonia UI. Dále jsme představili vývojové prostředí Microsoft Visual Studio a SQLiteStudio a přiblížili architekturu desktopové aplikace založenou na MVVM. Podrobně jsme rozebrali využití NuGet balíčky (Avalonia, ReactiveUI, EF Core, QuestPDF a další), které zajišťují robustnost, přenositelnost a reaktivitu řešení.

V praktické části práce jsme detailně popsali strukturu SQLite databáze se třemi tabulkami (Events, GlobalSettings, UserSettings), uspořádání projektové složkové struktury dle principu Separation of Concerns a implementaci prezentační vrstvy v Avalonia UI. Klíčovou součástí byla také realizace algoritmu pro vyvážení odpracovaných hodin, který se spouští při generování náhledu PDF a automaticky upravuje časy událostí tak, aby odpovídaly globálním či uživatelským nastavením.

Výsledná aplikace poskytuje uživatelsky přívětivé rozhraní pro rychlé zadávání a úpravy dat, vizualizaci denního, týdenního i měsíčního kalendáře a okamžitý náhled výsledného PDF dokumentu. Díky použití Avalonia UI běží plynule jak na Windows, tak na Linuxu, a díky Entity Framework Core se SQLite nabízí spolehlivé a přenosné datové úložiště.

Navržené řešení významně zkracuje čas potřebný k přípravě měsíčních a týdenních vyúčtování pracovní doby a minimalizuje riziko chyb při manuálním přepisování dat. Aplikace tak přispívá ke zvýšení administrativní efektivity a transparentnosti správy pracovního fondu na akademických pracovištích. V budoucnu je možné řešení rozšířit o jednotnou autentizaci (SSO), auditní logování či export do dalších formátů, což dále posílí jeho použitelnost a škálovatelnost.

POUŽITÁ LITERATURA

1. MICROSOFT, 2024. Jazyk C# – referenční příručka v češtině. In: learn.microsoft.com [online]. Microsoft, 2024 [cit. 2025-05-04]. Dostupné z: <https://learn.microsoft.com/cs-cz/dotnet/csharp/language-reference/>
2. SQLITE, 2025. Oficiální dokumentace SQLite. In: sqlite.org [online]. SQLite, 2025 [cit. 2025-05-04]. Dostupné z: <https://www.sqlite.org/lang.html>
3. AVALONIA UI, 2025. Dokumentace Avalonia UI. In: docs.avaloniaui.net [online]. Avalonia, 2025 [cit. 2025-05-04]. Dostupné z: <https://docs.avaloniaui.net/>
4. QUESTPDF, 2025. Oficiální dokumentace QuestPDF. In: questpdf.com [online]. QuestPDF, 2025 [cit. 2025-05-04]. Dostupné z: <https://www.questpdf.com/>
5. MICROSOFT, 2024. Entity Framework Core v .NET – práce s databázemi SQLite. In: learn.microsoft.com [online]. Microsoft, 2024 [cit. 2025-05-04]. Dostupné z: <https://learn.microsoft.com/cs-cz/ef/core/providers/sqlite/>
6. ARTIFEX SOFTWARE, 2025. Ghostscript – oficiální dokumentace. In: ghostscript.com [online]. Artifex Software, 2025 [cit. 2025-05-04]. Dostupné z: <https://www.ghostscript.com/>
7. HABJAN, John a kol., 2025. Ghostscript.NET – .NET wrapper pro Ghostscript. In: github.com/jhabjan/Ghostscript.NET [online]. Ghostscript.NET, 2025 [cit. 2025-05-04]. Dostupné z: <https://github.com/jhabjan/Ghostscript.NET>
8. AVALONIA XAML BEHAVIORS, 2024. GitHub – Avalonia.Xaml.Behaviors. In: github.com/AvaloniaXamlBehaviors [online]. Avalonia Team, 2024 [cit. 2025-05-04]. Dostupné z: <https://github.com/AvaloniaXamlBehaviors>
9. AVALONIA COMMUNITY, 2024. Práce s chováním (Behaviors) v Avalonia UI. In: docs.avaloniaui.net [online]. Avalonia Community, 2024 [cit. 2025-05-04]. Dostupné z: <https://docs.avaloniaui.net/docs/controls/behaviors>
10. REACTIVEUI COMMUNITY, 2024. ReactiveUI – dokumentace a postupy. In: reactiveui.net [online]. ReactiveUI Community, 2024 [cit. 2025-05-04]. Dostupné z: <https://www.reactiveui.net/docs/getting-started/>
11. MICROSOFT, 2023. Data binding a chování v Avalonia a WPF (Behaviors). In: learn.microsoft.com [online]. Microsoft, 2023 [cit. 2025-05-04]. Dostupné z: <https://learn.microsoft.com/cs-cz/dotnet/desktop/wpf/advanced/behaviors-overview>

SEZNAM PŘÍLOH

Příloha A: Zdrojový kód aplikace TeacherScheduleApp

PŘÍLOHA A: Zdrojový kód aplikace TeacherScheduleApp

Tato příloha obsahuje zip archiv s projektem z Visual Studio 2022 jménem TeacherScheduleApp